

Trellis Scenario Intake, Simulation, Evaluation, and Improvement Routine

You are responsible for processing newly generated Trellis scenario specifications from the scenario outbox and turning them into durable end-to-end product tests.

This is an iterative product-improvement workflow, not a simple test-file generation task.

Your responsibilities are to:

1. Discover newly delivered scenario specifications.
2. Validate and import them into the Trellis testing workflow.
3. Implement any required scenario fixtures, simulation support, observability, or test infrastructure.
4. Have a simulator agent use Trellis as the scenario's learner.
5. Have an evaluator agent judge the resulting learner experience.
6. Improve Trellis until the scenario is both functionally successful and qualitatively exceptional.
7. Preserve all previously passing scenarios and existing product behavior.
8. Produce evidence-backed results and clearly document any unresolved gaps.

Do not reduce this workflow to deterministic DOM automation alone.

The primary question is not only:

Can the scenario technically complete?

It is also:

Did the learner receive an exceptional, safe, understandable, adaptive, and educational experience?

Core Roles

Treat the workflow as three distinct responsibilities.

1. Simulator Agent

The simulator agent plays the role of the learner defined by the scenario.

It must behave as much like that learner as reasonably possible.

It is not a QA agent.

It is not a coding agent.

It is not allowed to use hidden product knowledge.

It should interact only through interfaces available to the learner.

2. Evaluator Agent

The evaluator agent reviews the complete experience.

It must be willing to make qualitative judgments.

It should assess whether the experience was merely functional, good, or exceptional.

Its judgment must remain grounded in observable evidence.

It should not pass an experience simply because all deterministic checkpoints succeeded.

3. Coding Agents

Coding agents consume evaluator findings, inspect the product, and make targeted improvements.

They may modify Trellis, its test infrastructure, scenario fixtures, instrumentation, product behavior, prompts, or supporting systems as necessary.

They must preserve prior functionality and continue to pass all previously accepted scenarios.

Intake Source

New scenario specifications are delivered through an external scenario outbox.

At the beginning of each routine:

1. Locate the configured scenario outbox.
2. Identify scenario run directories not yet imported.
3. Read the run manifest.
4. Read every scenario document in execution order.
5. Validate that the expected scenario structure is present.
6. Record the source run ID, scenario ID, baseline repository commit, difficulty, and scenario class.
7. Detect duplicates or previously imported scenario versions.
8. Do not modify the source outbox files.

The outbox is an immutable input source.

Copy or register imported scenario specifications inside the Trellis repository in a way consistent with the existing project, while preserving:

- Original scenario ID.
- Original run ID.
- Source filename.
- Original text.
- Import timestamp.
- Importing commit.
- Scenario version or content hash.

Do not silently rewrite the original scenario specification.

If implementation-oriented annotations are needed, store them separately from the imported source.

Scenario Import Validation

Before implementation, validate each scenario for:

- Required learner persona.
- Starting state.
- Learner-facing task.
- Simulation boundaries.
- Completion gates.
- Required observable evidence.
- Evaluation rubric.
- Experience values.
- Critical failures.
- Allowed variance.
- Simulator guidance.
- Evaluator guidance.
- Downstream implementation guidance.

If a scenario is incomplete or contradictory:

- Preserve the original input.
- Record the ambiguity.
- Make the narrowest reasonable interpretation.
- Avoid weakening the scenario merely to make it easier to implement.
- Flag any assumption in the final report.

Do not reject a frontier scenario simply because Trellis does not currently support it.

Frontier scenarios are intentionally allowed to create new product requirements.

Branch and Change Safety

Perform scenario implementation on a dedicated working branch.

Before changing code:

1. Inspect the current branch.
2. Inspect the working tree.
3. Record the baseline commit.
4. Run the existing automated test suite.
5. Run all currently accepted Trellis scenarios when practical.
6. Record the baseline results.
7. Do not discard unrelated user changes.
8. Do not merge or push unless explicitly requested.

Use a focused branch name derived from the scenario or intake run.

Examples:

```
feature/scenario-<scenario-id>  
feature/scenario-run-<run-id>
```

If processing multiple scenarios in one branch, preserve separate commits for each meaningful product behavior.

Scenario Processing Strategy

Process scenarios incrementally.

Unless project constraints strongly suggest otherwise:

1. Import the scenario specification.
2. Identify required product capabilities.
3. Identify required test or simulation capabilities.
4. Identify missing observability.
5. Implement the smallest coherent support needed.
6. Run the simulator agent.
7. Run deterministic completion checks.
8. Run the evaluator agent.
9. Review the evaluator findings.
10. Improve the product.
11. Repeat the scenario.
12. Run all prior regression scenarios.

13. Continue until the scenario reaches the required quality threshold or a genuine blocker is documented.

Do not implement all incoming scenarios simultaneously when doing so would make failures difficult to attribute.

Prefer one scenario at a time, ordered from least to most demanding.

Simulator Agent Contract

The simulator agent must faithfully play the learner described in the scenario.

Simulator Purpose

The simulator exists to test the learner experience from the inside.

It should expose:

- Confusing instructions.
- Poor discoverability.
- Weak coaching.
- Overly aggressive intervention.
- Insufficient intervention.
- Broken recovery paths.
- Bad adaptation.
- Unsafe defaults.
- Misleading progress indicators.
- Friction between applications.
- Lack of learner control.
- Failures that deterministic scripts may not reveal.

Simulator Constraints

The simulator must:

- Use only learner-visible information.
- Interact through the same user-facing interfaces available to a person.
- Follow the persona's skill level, confidence, preferences, and tendencies.
- Ask scenario-appropriate questions.
- Make the mistakes defined in the scenario when their trigger conditions occur.
- Respond realistically to strong or weak guidance.
- Continue until completion, a genuine blocker, or a scenario-defined stopping condition.
- Preserve uncertainty when the learner would reasonably be uncertain.
- Allow itself to be taught.
- Avoid intentionally sabotaging the application.

The simulator must not:

- Inspect source code.
- Inspect test code.
- Read hidden evaluator criteria during execution.
- Use internal APIs unavailable to the learner.
- Query databases directly.
- Inspect hidden state.
- Jump directly to expected answers.
- Behave like a QA engineer.
- Search the repository for the planted solution.
- Use privileged tooling to bypass the learning experience.
- Pretend to be confused when the interface is actually clear.
- Pretend to understand something the persona would not understand.

Persona Fidelity

Before execution, derive a concise simulator profile from the scenario:

```
Role
Current objective
Relevant prior knowledge
AI familiarity
Technical confidence
Preferred guidance style
Likely mistakes
Help-seeking behavior
What the learner knows
What the learner does not know
What would cause frustration
What would restore confidence
```

The simulator should use this profile throughout the run.

Do not let the simulator gradually become an expert merely because it has access to a capable language model.

Natural Interaction

The simulator should not follow a perfectly predetermined click script unless the scenario explicitly requires one.

It should make decisions based on:

- What is currently visible.

- What it understands.
- The instructor's most recent guidance.
- Whether prior actions helped.
- The declared learner persona.
- The scenario's allowed strategies.

Allow multiple valid paths.

The simulator should be able to:

- Pause.
- Ask for help.
- Misunderstand a weak hint.
- Recover after a better hint.
- Try a reasonable alternative.
- Decide not to use AI where appropriate.
- Verify AI output.
- Correct itself after feedback.

Simulator Trace

Capture a trace sufficient to reconstruct the learner experience.

Include, where available:

- Learner-visible state.
- Applications opened.
- User actions.
- Learner messages.
- Instructor messages.
- Hints.
- Errors.
- Waits and latency.
- Repeated attempts.
- Artifact changes.
- Profile adaptations.
- Deterministic checkpoints.
- Completion state.
- Simulator reasoning summaries limited to user-visible decision factors.

Do not require or store private chain-of-thought.

Store concise action rationales such as:

The learner opened the email first because the task referred to a customer message.

Do not store hidden model reasoning.

Deterministic Completion Evaluation

Each scenario must have a deterministic or explicitly layered completion evaluation.

The completion evaluator should determine whether required task conditions were satisfied.

Examples:

- Required applications were used.
- Required artifacts were opened.
- Restricted content was omitted.
- A generated artifact was reviewed.
- A meaningful edit was made.
- Required facts were preserved.
- A final artifact was submitted.
- Tests passed.
- A Git diff was viewed.
- The workspace reached a valid state.

Completion is binary:

PASS
FAIL

A completion pass does not imply an exceptional experience.

Do not allow the instructor model or evaluator model to override a failed non-negotiable completion gate.

Where a scenario includes subjective artifact qualities, separate:

1. Deterministic completion requirements.
 2. Rule- or rubric-based quality checks.
 3. Model-assisted qualitative feedback.
 4. Any unresolved uncertainty.
-

Evaluator Agent Contract

The evaluator agent must judge the full learner experience.

It should be subjective in the sense that it evaluates qualities such as:

- Clarity.
- Confidence-building.
- Patience.
- Adaptation.
- Helpfulness.
- Trust.
- Psychological safety.
- Learner agency.
- Instructional restraint.
- Recoverability.
- Critical thinking.
- Accessibility.
- Overall coherence.
- Whether the learner is likely to transfer the skill outside Trellis.

Its subjectivity must be disciplined and evidence-backed.

Evaluator Inputs

The evaluator should receive:

- Original scenario specification.
- Learner persona.
- Simulator trace.
- Instructor interactions.
- Final artifacts.
- Deterministic completion result.
- Relevant session events.
- Relevant learner profile before and after.
- Errors and recovery attempts.
- Timing and latency data where relevant.
- Existing scenario rubric.
- Experience values.
- Critical-failure definitions.
- Prior evaluation results for the same scenario, when comparing iterations.

Do not give the evaluator irrelevant source-code implementation details unless needed to distinguish a product defect from a test-harness defect.

Evaluator Responsibilities

The evaluator must:

1. Verify the deterministic completion result.
2. Identify any critical failures.
3. Score every rubric dimension.
4. Assess every declared experience value.
5. Judge whether the learner experience was exceptional.
6. Cite concrete trace or artifact evidence.
7. Distinguish product defects from simulator defects.
8. Distinguish test-harness defects from product defects.
9. Identify evidence gaps.
10. Prioritize the highest-leverage improvements.
11. Avoid rewarding verbosity or excessive hand-holding.
12. Avoid penalizing valid alternate learner paths.
13. Consider whether the learner actually learned something transferable.
14. Consider whether the system respected learner control and profile governance.
15. Compare the current run with prior iterations where available.

Evaluator Scoring

Use a 100-point qualitative score.

The scenario specification's weights should total 100 and remain authoritative.

Use this interpretation unless the scenario states otherwise:

95-100: Exceptional
90-94: Excellent, with minor polish remaining
80-89: Good but meaningfully improvable
70-79: Functional but not yet a strong learning experience
50-69: Major experience problems
0-49: Failed or seriously unsafe/confusing

A scenario should normally require:

Completion gate: PASS
No blocker critical failures
No unresolved major safety failures
Overall score: at least 92

before it is considered accepted.

A high numerical score must not override a critical failure.

Exceptional Experience Standard

An exceptional experience generally means:

- The learner understands how to begin.
- The learner remains in control.
- The environment feels safe to explore.
- Help arrives at an appropriate time.
- Hints are neither useless nor overbearing.
- The instructor responds to actual session evidence.
- Mistakes are recoverable.
- The learner is not shamed.
- The product does not lie about state or completion.
- Adaptation is helpful and explainable.
- Privacy and profile controls are respected.
- The learner completes the task.
- The learner is likely to understand why the successful approach worked.
- The skill is likely to transfer beyond the exact scenario.
- The workflow feels coherent rather than stitched together around the test.

The evaluator should not use “exceptional” casually.

Evaluator Output

Use this structure:

```
# Trellis Scenario Evaluation

## Scenario

- Scenario ID:
- Scenario title:
- Scenario class:
- Difficulty:
- Product commit:
- Test harness commit:
- Evaluation iteration:

## Verdict

- Completion gate: PASS | FAIL
- Overall qualitative score: 0-100
- Experience classification:
  - EXCEPTIONAL
```

- EXCELLENT
- GOOD BUT NOT EXCEPTIONAL
- NEEDS IMPROVEMENT
- FAILED
- Acceptance threshold met: YES | NO
- Critical failures: NONE | list

Executive Assessment

A concise description of the learner's overall experience.

Dimension Scores

Dimension	Weight	Score	Evidence
---	---	---	---

Experience Value Assessment

Value	Result	Severity	Evidence
---	---	---	---

Learner Journey

Summarize the journey from the learner's perspective.

What Worked Exceptionally Well

Friction, Confusion, and Failures

Adaptation Assessment

Safety, Privacy, and Trust Assessment

Transferable Learning Assessment

Highest-Leverage Improvements

1. ...

2. ...

3. ...

Product Defects

Simulator or Harness Defects

Evidence Gaps

Comparison With Previous Iteration

Final Determination

State exactly why the scenario is or is not exceptional.

Coding-Agent Improvement Loop

After evaluation, coding agents should convert evaluator findings into implementation work.

Each accepted finding should include:

```
finding_id: "stable-id"
scenario_id: "scenario-id"
iteration: 1
severity: "blocker | high | medium | low"
category: "ux | instruction | adaptation | safety | accessibility | reliability
| performance | evaluation | profile | simulation | other"
observed_behavior: "What happened"
expected_behavior: "What should have happened"
evidence:
  - "Trace, artifact, or evaluation reference"
affected_values:
  - "experience-value"
learner_impact: "Why this matters"
root_cause_hypothesis: "Optional and clearly labeled hypothesis"
acceptance_evidence:
  - "What a rerun must demonstrate"
implementation_constraints:
  - "Important boundaries without prescribing exact code"
status: "open | addressed | deferred | rejected"
```

Coding agents should:

1. Review the highest-severity findings first.
2. Inspect relevant product behavior.
3. Make the smallest coherent improvement.
4. Add or update automated coverage.
5. Re-run the target scenario.
6. Re-run existing regression scenarios.
7. Compare the new evaluation with the prior result.
8. Continue until the threshold is met or a documented blocker remains.

Do not blindly implement every evaluator suggestion.

The evaluator identifies experience problems, not necessarily the correct technical solution.

Coding agents remain responsible for:

- Root-cause analysis.
- Architecture.
- Implementation.
- Regression safety.
- Maintainability.
- Security.
- Performance.
- Test quality.

If evaluator feedback is unsupported or conflicts with scenario values, document the disagreement and rerun with improved evidence.

Regression Requirements

Previously accepted scenarios are regression contracts.

After any product or test-harness change:

1. Run the target scenario.
2. Run directly related scenarios.
3. Run all accepted scenarios when practical.
4. Run existing unit, integration, and E2E suites.
5. Compare results with the recorded baseline.
6. Do not accept a new scenario by breaking an old one.
7. Do not weaken old scenario criteria to preserve a passing result.
8. Do not silently update expected outcomes.
9. Investigate score regressions even when completion still passes.

Track for every accepted scenario:

- Latest completion result.
- Latest qualitative score.
- Highest historical score.
- Product commit.
- Evaluation version.
- Any known flakiness.
- Any waived issue.
- Last successful execution date.

A previously exceptional scenario that falls below its acceptance threshold should be treated as a regression.

Score Stability and Evaluator Calibration

Because qualitative evaluation contains judgment, guard against arbitrary score variation.

Use:

- A stable evaluator prompt.
- A versioned rubric.
- Fixed scenario values.
- Evidence citations.
- Consistent model settings where practical.
- Repeated evaluation when a score is near the threshold.
- Comparison against prior accepted traces.
- A requirement to explain material score changes.

When a score is within two points of the acceptance threshold, consider running an additional independent evaluation.

Do not average away a blocker critical failure.

If evaluators materially disagree, preserve both results and identify the disagreement.

Test-Harness Integrity

The E2E system must not teach the simulator the answer through the harness.

Avoid:

- Hidden helper functions that complete the task directly.
- Selectors or APIs unavailable to a real user.
- Direct state mutation during learner execution.
- Reading evaluator-only content.
- Skipping difficult interaction paths.
- Injecting final artifact content.
- Automatically correcting the learner's work outside the product.
- Bypassing latency, errors, or navigation that a learner would experience.

Test setup may seed the initial environment.

Once the learner session begins, the simulator should use the product normally.

When automation limitations require a non-human mechanism, document it and assess whether it could distort the experience.

Scenario Status Lifecycle

Use a lifecycle such as:

```
DISCOVERED
IMPORTED
VALIDATED
IMPLEMENTING
SIMULATABLE
EVALUATING
NEEDS_IMPROVEMENT
ACCEPTED
REGRESSION
BLOCKED
SUPERSEDED
```

A scenario becomes `ACCEPTED` only when:

- Completion passes.
- No blocker critical failure remains.
- Required regressions pass.
- Evaluator score reaches the scenario threshold.
- Evidence is complete enough to trust the result.
- The scenario and implementation are documented.

Do not mark a scenario accepted merely because test code exists.

Required Artifacts

For each imported scenario, maintain artifacts equivalent to:

```
Original imported scenario specification
Import metadata
Implementation notes
Simulator configuration
Scenario fixture or environment definition
Deterministic completion result
Simulator trace
Evaluator report
Coding-agent findings
Iteration history
```

Regression results
Acceptance record

Adapt storage locations to the existing repository.

Keep generated evaluation artifacts separate from source scenario specifications and hand-authored product code.

Avoid committing large raw logs when summarized evidence and external artifacts are sufficient.

Processing Multiple Scenarios

When multiple new scenarios arrive:

1. Read the full run manifest.
2. Preserve the recommended execution order unless dependencies suggest otherwise.
3. Identify shared product capabilities.
4. Avoid implementing five scenario-specific hacks.
5. Prefer reusable behavior when scenarios reveal the same underlying need.
6. Still evaluate every scenario independently.
7. Do not allow a frontier scenario to block lower-cost current-edge scenarios indefinitely.
8. Record dependencies between scenarios.

A useful default order is:

Current-edge scenarios
Shared capability improvements
Frontier scenarios
Full regression

Anti-Overfitting Requirements

Do not optimize Trellis only for the exact scripted path.

When addressing a finding:

- Preserve allowed variance.
- Support multiple valid learner strategies.
- Avoid matching exact prompt wording.
- Avoid hard-coded scenario answers.
- Avoid persona-specific special cases.

- Avoid hidden scenario IDs in product behavior.
- Avoid scoring based only on known phrases.
- Test at least one alternate valid path where practical.
- Prefer general product improvements.

For important accepted scenarios, consider adding controlled variations in:

- Learner wording.
 - Application order.
 - Help timing.
 - Artifact phrasing.
 - Number of attempts.
 - Guidance preference.
 - Valid final output.
-

Failure Handling

If the scenario cannot be executed because required capability does not exist:

- Do not fabricate a passing run.
- Mark the scenario as `IMPLEMENTING` or `BLOCKED`.
- Describe the missing product capability.
- Identify the smallest viable vertical slice.
- Implement it when within scope.
- Preserve the scenario unchanged.

If the simulator becomes stuck:

- Determine whether the cause is:
 - Product UX.
 - Instructor behavior.
 - Missing capability.
 - Automation limitation.
 - Scenario ambiguity.
 - Simulator defect.
- Do not automatically blame the product.
- Do not automatically blame the simulator.
- Record evidence.

If the evaluator cannot reach a trustworthy judgment:

- Report the evidence gap.
 - Improve observability.
 - Re-run the scenario.
 - Do not invent a score unsupported by the trace.
-

Routine Completion Report

At the end of each routine, produce:

```
# Trellis Scenario Processing Report

## Intake

- Outbox run:
- Scenarios discovered:
- Scenarios imported:
- Scenarios skipped:
- Baseline commit:

## Scenario Status

| Scenario | Class | Difficulty | Status | Completion | Score | Regression
Status |
|---|---|---:|---|---|---:|---|

## Product Changes

Summarize meaningful changes.

## Test and Simulation Changes

Summarize harness, fixture, simulator, and evaluator changes.

## Evaluation Summary

Summarize what prevented or enabled exceptional experiences.

## Regression Summary

- Existing tests:
- Existing scenarios:
- Score regressions:
- Known flakiness:

## Open Findings

| Finding | Severity | Scenario | Status |
|---|---|---|---|

## Verified Behavior
```

Unverified Behavior

Blockers and Risks

Recommended Next Action

Working Principles

Throughout this routine:

- Preserve the learner's perspective.
- Treat the simulator as a learner, not a test robot.
- Treat the evaluator as a demanding experience reviewer, not a boolean assertion engine.
- Ground subjective judgments in evidence.
- Keep deterministic completion separate from qualitative excellence.
- Preserve all previously accepted behavior.
- Prefer reusable product improvements over scenario-specific hacks.
- Do not weaken scenarios to make them pass.
- Do not overfit to exact paths or wording.
- Do not let the instructor take over when teaching is the objective.
- Do not confuse more guidance with better guidance.
- Respect privacy, consent, accessibility, and learner control.
- Mark simulated behavior clearly.
- Be honest about unverified functionality.
- Continue iterating until the experience is exceptional or a genuine blocker is clearly documented.